

Minimizing the total travel distance for the locker-based drone delivery: A branch-and-cut-based method

Waiming Zhu^{a,b}, Xiaoxuan Hu^{a,b,c,*}, Jun Pei^{a,c}, Panos M. Pardalos^d

^a School of Management, Hefei University of Technology, Tunxi road, Hefei, 230009, Anhui, China

^b Anhui Aerospace System Intelligent Management Engineering Research Center, Tunxi road, Hefei, 230009, Anhui, China

^c Key Laboratory of Process Optimization and Intelligent Decision-making, Ministry of Education, Tunxi road, Hefei, 230009, Anhui, China

^d Center for Applied Optimization, Department of Industrial and Systems Engineering, University of Florida, Gainesville, FL, USA

ARTICLE INFO

Keywords:

Drone delivery problem
Locker-based drone delivery
Routing and parking hybrid scheduling
Total unimodality

ABSTRACT

This article investigates the problem of minimizing the total travel distance in locker-based drone delivery, where the roofs of lockers are reused as parking platforms for drones. It is a drone routing and parking hybrid problem with modeling challenges. We find the sufficient and necessary conditions for feasible solutions and transform the original problem into a scale-tractable one. Subsequently, we propose a compact lower-bound formulation for the transformed problem and prove the total unimodality of the coefficient matrix. Furthermore, we develop a two-stage method in which a branch and cut algorithm solves the transformed problem and a heuristic constructs practical schedules for the original problem. Simulated tests demonstrate that the method can solve each simulated instance within one second. Random tests reveal that the method can efficiently solve instances with 1000 sites and 1500 tasks within an acceptable CPU time. A sensitivity analysis indicates that the complexity arising from routing flexibility is greater than that arising from parking flexibility.

1. Introduction

Over the past few years, the commercial employment of logistical drones has witnessed a remarkable surge in popularity worldwide. Several companies, including Amazon, Google, UPS, Alphabet, Zipline, DHL, Alibaba, and Jindong, are eagerly entering the market of drone delivery (Sah et al., 2021). The market for this innovative service is expected to grow \$100 billion by the end of the next decade (Liu and Sun, 2022).

In locker-based drone delivery (LDD) (Franco, 2016), locker roofs are reused as drone parking platforms, enabling a more coordinated and cooperative approach. Fig. 1 illustrates this unmanned aerial delivery process, where a unit-capacity drone loads a parcel on a source locker and later unloads it at the destination locker upon landing. This process, encompassing loading, travel, and unloading, is highly automated and remotely monitorable. As such, LDD holds significant potential, particularly in logistical environments with minimal physical contact requirements (Nisingizwe et al., 2022). Two Chinese companies, Antwork and Meituan, have successfully implemented this model in food delivery and medical transportation scenarios (Liang, 2019; Li et al., 2022). The cumulative travel distance has exceeded tens of thousands of kilometers.

As the deployment scale of LDD systems continues to increase, the requirements for coordinating drones and lockers will become increasingly urgent. Given a set of parcels in an LDD system with multiple lockers and drones, transportation schedules need to be created to ensure the completion of delivery tasks. Generally, a drone's transportation cost is approximately proportional to its travel

* Corresponding author at: School of Management, Hefei University of Technology, Tunxi road, Hefei, 230009, Anhui, China.

E-mail addresses: zhuwaiming@hfut.edu.cn (W. Zhu), xiaoxuanhu@hfut.edu.cn (X. Hu), peijun@hfut.edu.cn (J. Pei), pardalos@ufl.edu (P.M. Pardalos).

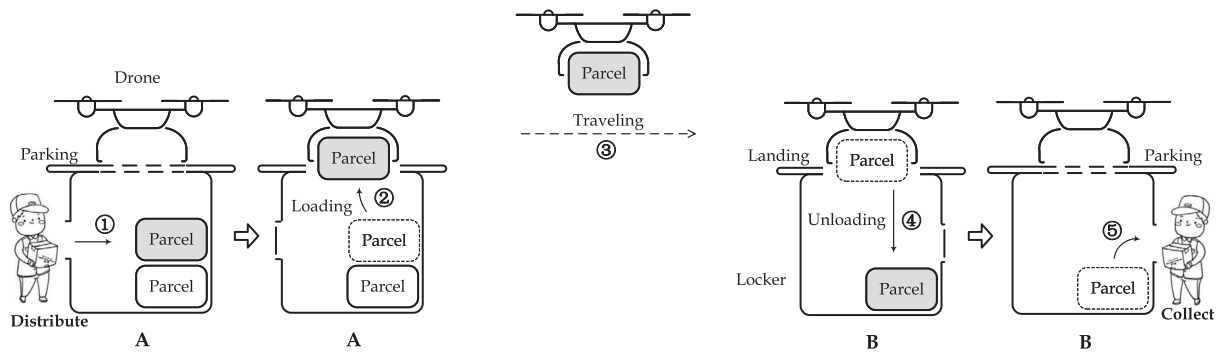


Fig. 1. An illustration of locker-based drone delivery.

distance. To reduce costs, we put forward an optimization problem with the goal of minimizing total travel distance of drones. It is worth noting that, to the best of our knowledge, this study represents the first attempt to solve the LDD problem (LDDP).

The LDDP is a combinatorial optimization problem that combines routing and parking sub-problems corresponding to the vehicle routing problems (VRPs) (Laporte, 2009; Pessoa et al., 2021; Ledvina et al., 2022) and machine scheduling problems (Hall and Posner, 2001), respectively. Modeling the LDDP with tight formulations can be challenging.

First, straightforward formulation techniques can hardly be used. The reason is that we need to use variables to represent the drone routes. These routes comprise segments for both loaded flight legs and empty flight legs, where the empty flight legs are used for drone repositioning. However, the details of empty flight legs remain unknown until the problem is resolved. Second, trip-based concise formulations, typically employed in VRPs, are rarely applicable due to the hybrid nature of the parking sub-problem. Alternative improvement models can be used, e.g., the trip-based time-index formulation (TTIF) and trip-based two-stage formulation (TTSF). However, both formulations are difficult to solve.

Inspired by the famous traveling salesman problem (TSP) formulation proposed by Dantzig et al. (1954), we aimed to model the LDDP using similarly simple but elegant formulations. The main contributions of this article are as follows.

- This article presents the sufficient and necessary conditions for a solution to be feasible in the LDDP. The main condition pertains to the relationship between the state of the delivery system and the degree difference in the digraph constructed based on the flight legs of the solution.
- Based on the conditions, a transformed problem is obtained. Lower bound (LB) and upper bound (UB) formulations are developed for the transformed problem. The coefficient matrix of the LB formulation is proved to be totally unimodular and hence, integral solutions can be obtained using a linear programming (LP) method.
- A branch-and-cut-based two-stage method is developed for the LDDP. In the first stage, a branch and cut (B&C) algorithm is developed to solve the transformed problem. In the second stage, a heuristic is developed to construct practical schedules for the original problem. The two-stage method can compute optimal solutions efficiently.

Three sets of simulated instances are used to verify the strengths of the B&C algorithm. These sets are constructed to simulate scenarios involving food delivery, sample submission, and instant delivery. We collected real location and distance data from nine Chinese cities, resulting in a dataset of 81 instances. Each instance is solved to optimality within 1 s. The results show that the travel distance of necessary empty flight legs depends significantly on the type of delivery scenario.

The numerical tests are conducted on two sets of random instances. The first set comprises 30 instances generated following a uniform distribution, with the largest instance containing 1000 sites and 1500 tasks. The algorithm successfully solves each of these instances to optimality within 50 s. The second set consists of 40 “hard” instances generated following specific rules. The algorithm solves each of these instances to near optimality (Gap < 1%) within 100 s. The results show that the number of cuts is significantly greater than that of B&B nodes, highlighting the integral merits of the formulation.

A sensitivity analysis for the number of drones is also conducted on ten selected “hard” instances. The results show that the instances with fewer drones are more difficult to be solved. Therefore, the difficulty caused by routing flexibility is stronger than that caused by parking flexibility. Overall, the proposed method is highly promising for the problem.

The remainder of this paper is organized as follows. Section 2 provides a literature review. Section 3 defines the LDDP in detail. Section 4 introduces the sufficient and necessary conditions. Section 5 introduces the two-stage method. Section 6 describes the computational tests. Section 7 concludes the article.

2. Literature review

To the best of our knowledge, this is the first study on the LDDP. Therefore, we review the literature about related problems, encompassing the collaborative truck–drone delivery problems, the integrated ship routing and berth allocation problems (ISRBAPs), and so on.

2.1. Collaborative truck–drone delivery problems

We harvest a large number of literature on drone delivery problems. However, very close literature are surprisingly sparse. Most articles investigate the following two problems. The first problem is the parallel drone scheduling TSP (PDSTSP) (Dell’Amico et al., 2020; Raj et al., 2021; Lei and Chen, 2022; Saleu et al., 2022; Nguyen et al., 2022; Chen et al., 2022) in which trucks and drones are used to execute delivery tasks concurrently. Both the trucks and drones depart from and return to a central depot, operating independently. Each truck travels single tour while serves multiple customers. Whereas a drone takes multiple trips and serves one customer per trip. It is important to note that each drone strictly follows a back-and-forth pattern to execute its delivery tasks. The second problem encompasses the TSP with drone (TSP-D) and its extension, the VRP with drones (VRP-D) (Murray and Chu, 2015; Wang et al., 2017; Carlsson and Song, 2018; Poikonen et al., 2019; Poikonen and Golden, 2020; Agatz et al., 2018; Ha et al., 2018; Faiz et al., 2024; Zhen et al., 2022a; Tiniç et al., 2023). In those scenarios, trucks and drones work in tandem to serve customers. Trucks depart from a depot carrying parcels and drones on board. Initially, the drone rides on the truck and then launches from the truck to perform its own parcel deliveries during the trip. Once a delivery is completed, the drone returns to the truck. This collaborative mode enables the drone to serve remote customers.

Those problems aim at developing practical plans for both trucks and drones with certain objectives. The popular studied objectives are minimizing the makespan (Murray and Chu, 2015; Wang et al., 2017; Carlsson and Song, 2018; Poikonen et al., 2019; Poikonen and Golden, 2020) and the cost (Agatz et al., 2018; Ha et al., 2018; Faiz et al., 2024; Zhen et al., 2022a; Tiniç et al., 2023). Solving these problems is more challenging than the traditional TSPs or VRPs. Generally, the routing network of the drones is different from that of the trucks. The problems essentially make routes on a hybrid network for two echelon vehicles (Faiz et al., 2024). A variety of algorithms are put forward for these problems, including branch-and-bound (Poikonen et al., 2019; Poikonen and Golden, 2020), branch-and-cut (Wang and Sheu, 2019; El-Adle et al., 2021; Tamke and Buscher, 2021; Tiniç et al., 2023), branch-and-price (Zhen et al., 2022a; Yang et al., 2023; Roberti and Ruthmair, 2021; Zhou et al., 2023), and heuristics (Murray and Chu, 2015; Ha et al., 2018; Jeong et al., 2019; Dayarian et al., 2020). For comprehensive surveys, we refer the readers to Otto et al. (2018) and Li et al. (2021).

Compared to the LDDP, these problems do not have to address the parking issue. In the PDSTSP, a drone launches from and returns to the depot to execute each delivery task. The depot has sufficient parking platforms for the drones. Therefore, parking conflicts are disregarded. In the TSP-D or VRP-D, a drone is associated with a dedicated truck that is always available for the drone’s parking. Therefore, parking conflicts are disregarded as well. However, in the LDDP, the drone needs to travel between lockers, potentially leading to parking congestion. Therefore, we need to schedule the parking precedence for each locker. The focus of the PDSTSP is to trade off the utilization of two-echelon vehicles for delivery tasks. The TSP-D and the VRP-D are challenging optimization problems with extreme routing flexibility. However, the LDDP is a routing and parking hybrid problem.

2.2. Integrated ship routing and berth allocation problem

The LDDP is also related to the ISRBAPs. Ship routing seeks to make routes for vessels and transshipment for cargos such that operational costs are minimized (Wang et al., 2013; Meng et al., 2015; Boland et al., 2017; Li et al., 2019; Wu et al., 2021). The vessels and cargos are analogous to the drone and the parcel in the LDDP, respectively. Berth allocation seeks to assign the time periods of berths to vessels such that all the vessels are served at quays, and operational costs are minimized (Wang et al., 2018; Jia et al., 2020a, 2022; Zhen et al., 2022b; Guo et al., 2023). One critical constraint is that each berth serves only one vessel at a time. Schedule must be specified for the berths to avoid loading or unloading conflicts. Therefore, the berth is analogous to the locker in the LDDP.

However, the ship routing and berth allocation are usually separately investigated. Only a few works study the integrated problem. Li and Pang (2011) study such an ISRBAP in which cargo transshipment is disallowed. They formulate the problem with a mixed integer linear program and develop a valid inequality to strengthen the model. They also put forward a variable-relaxed initial solution. The potential cost-saving benefits attainable by integrated planning is verified by numerical tests. Pang and Liu (2014) investigate an ISRBAP in which transshipment is allowed. They put forward a two-stage heuristic to solve the problem. In the first stage, the routes of vessels are computed without considering the transshipment. In the second stage, the computed routes are improved by considering transshipment and the berth allocation plan are determined. The results of computational experiments show that integrated planning is superior to sequential and independent planning.

There are differences between the studied ISRBAP and the LDDP. Firstly, the vessels are allowed to wait at sea without cost in these ISRBAPs. As comparison, in the LDDP, there is a strict limitation that a drone must be served once it arrives at a locker. A drone can hardly wait in the air because it consumes power energy when it is flying. Accordingly, constructing feasible solutions is even more difficult. Secondly, in the studied ISRBAPs, a vessel is not allowed to visit a terminal more than once. While in the LDDP, a drone may visit a locker by several times. The multiple visits of drones need an unfixed number of variables. Thus, modeling the LDDP becomes even more difficult as well. These differences suggest that the methods of the ISRBAP can hardly be applied for the LDDP in a straightforward way.

2.3. Other related problems

There are some other problems that relate to the LDDP. Firstly, we review the aircraft routing problems in airlines (Barnhart et al., 1998; Gopalan and Talluri, 1998; Liang and Chaovaitwongse, 2013; Liang et al., 2015). Those problems aim at determining practical routes for a fleet of aircraft such that each given flight leg is covered by one aircraft and a total cost is minimized. The flight legs and aircraft are analogous to the delivery tasks and the drones in this article, respectively. Thus, those problems are highly related to the LDDP. The main difference is that there is a loose parking limit at the airports in those problems. Some scholars investigate the aircraft routing and crew scheduling integrated problems (Cordeau et al., 2001; Sandhu and Klabjan, 2007; Ruther et al., 2017). The crew scheduling assigns a set of crews to the flight legs and makes a personalized weekly or monthly schedule for each individual crew. The hybrid nature of the integrated problems is similar to the feature of the LDDP. However, the crew scheduling is conducted on given timetables of flight legs. Namely, the time points are discrete. While in the LDDP, there is no such straightforward time points for drones and lockers.

Secondly, we examine the seaport service planning problem with consideration of pilots (Jia et al., 2020b; Wu et al., 2022). These problems make integrated plans for vessels, berths, and pilots to mitigate congestion in seaports. Such a plan contains a set of pilots' routes which are similar to the drone routes in the LDDP. A pilot's navigation operation is analogous to a drone's loaded flight leg in which a parcel is delivered. A pilot's reposition operation is analogous to a drone's empty flight leg in which no parcel is delivered. However, there are rare parking limits for the pilots in the standpoints of the seaport. Thus, these problems are also different from the LDDP.

Lastly, we review the parking management problems (Yang et al., 2013; Liu et al., 2014; Jian et al., 2020). Those problems make managing policies for vehicle parking actions at a strategic level. For example, Yang et al. (2013) find that an appropriate combination of reserved and unreserved parking spots can temporally relieve traffic congestion. Liu et al. (2014) further propose a parking reservation scheme with expiration times. Jian et al. (2020) propose that integrating car-sharing and parking-sharing services can probably improve the operator's profit and social welfare. In comparison, the LDDP aims at making practical plans for the drones and lockers at an operational level.

3. Locker-based drone delivery problem

In this section, we will start by introducing the LDDP and then explore the challenges of modeling the problem.

3.1. Problem definition

The problem is defined on a set of logistical sites V that are scattered throughout a city. At each site $v \in V$, a total of c_v lockers are deployed to automatically collect and distribute parcels. The roofs of lockers are repurposed as parking platforms for identical drones. Due to the limited roof space, each locker's parking platform can accommodate only one drone at a time, as shown in Fig. 1. Concurrent parking of multiple drones at the same locker is not allowed. A locker must serve the drones one by one. As a result, the limited number of lockers becomes a shared parking resource for drones.

There is a set of tasks T needing to be delivered. Each task consists of a non-splittable parcel that can be fully loaded by a drone. Generally, the locker is relatively large and the parcel is small. Therefore, we assume that the locker can hold as many parcels as possible. Accordingly, the parcel packing problem inside the locker is disregarded. Each task $t \in T$ is associated with a source site n_t and a destination site q_t . Note that the parcels are not loaded at a central depot. The tasks in the LDDP are analogous to the users in the dial-a-ride problem (Cordeau and Laporte, 2007). In the LDDP, repetitive tasks with the same source and destination sites are allowed. In other words, it is possible for equations $n_t = n_{t'}$ and $q_t = q_{t'}$ to be true for any $t \neq t', t \in T, t' \in T$. Therefore, the drones may visit a locker multiple times.

Each drone $j \in J$ has a unit capacity due to its limited lifting ability. This nature is also introduced in the "flying sidekick" (Murray and Chu, 2015) and "horsefly routing" (Carlsson and Song, 2018) programs. At the start of the delivery process, drones are parked on certain lockers. Subsequently, the drones load parcels and take off from the lockers at the source sites. The drones travel to the lockers at the destination sites and unload the parcels after landing. The loading, travel, and unloading operations of the drones are fully automated.

This article assumes that the drone's flying range is large enough that any site $v' \in V$ is reachable for the drone at site $v \in V$. This is reasonable, as some real-world drones can travel more than one hundred miles, which exceeds the diameter of a regular city. A drone can get its battery swapped each time it performs a landing operation on a locker. Therefore, the power supply schedule is not considered in this article. Furthermore, relay delivery is not allowed in current applications. More definitely, a drone carrying a parcel cannot travel across more than one successive airways.

A straight-line airway is associated with each pair of sites. The length of an airway is computed by the Euclidean distance between the two sites. Since lockers located at the same site are very close to each other, the distance traveled by a drone between lockers at the same site is negligible. A drone can maneuver among the lockers at the same site to load parcels as long as there are no parking conflicts.

Operating costs are computed based on consumed resources, such as power energy, which exhibit an approximate linear relationship with the drone's travel distance or travel time. It is important to note that minimizing the total travel distance is equivalent to minimizing the total travel time if no time windows are imposed on tasks. Therefore, in order to efficiently execute all the tasks, we put forward the LDDP, which is defined as below.

Definition 1. The Locker-Based Drone Delivery Problem (LDDP): Given an LDD system and a set of tasks, the problem involves selecting a set of successive flight legs for each drone and determining the start and stop times for each takeoff and landing operations of the flight legs, ensuring that all tasks are completed, there are no parking conflicts at each locker, and the total distance traveled by the drones is minimized.

There are two types of flight legs. The first type is **loaded** flight legs where a drone carries a parcel. Since relay delivery is not allowed, one loaded flight leg needs to be arranged for each task. The number of loaded flight legs is fixed and equals to the number of tasks ($|T|$). Consequently, the total distance of all loaded flight leg is also fixed. The second type is **empty** flight legs where a drone travels without a parcel. The purpose of arranging empty flight legs is to facilitate the connection between the loaded flight legs. For example, if there is no drone parked on a source locker, a drone should be acquired from other lockers before the task can be executed. Conversely, if a drone is parked on a destination locker, it should be moved away to provide an idle parking platform. Clearly, the empty flight legs of optimal solutions are unknown before solving the problem. In fact, the LDDP can be transformed into a problem that aims to create feasible transportation plans, with the objective of minimizing the total distance of the empty flight legs.

The LDDP is a routing and parking hybrid problem with solutions consisting of two inseparable sub-schedules: travel routes for drones and serving schedules for lockers. The routes-making sub-problem shares several features with the multi-trip VRP (Paradiso et al., 2020). The locker-scheduling sub-problem shares several features with the machine scheduling problems (Hall and Posner, 2001). These two sub-problems cannot be solved independently. On the one hand, the locker parking schedules are the optimized output of the routing sub-problem. On the other hand, feasible drone routes can only be determined using parking status as input. In other words, different routes result in different parking schedules and vice versa. Therefore, the LDDP is a routing and parking hybrid problem.

3.2. Analysis of modeling and solving challenges

A straightforward modeling method uses variables to represent all possible routes and serving orders. When modeling the problem using this method, empty flight legs must be included as inputs. Otherwise, the drone routes cannot be represented completely. However, as previously discussed, the empty flight legs in optimal solutions is unknown prior to solving the problem. Therefore, the straightforward modeling method is not suitable for the LDDP.

One possible strategy is to represent the routes implicitly. For example, trip-based formulations (Desaulniers et al., 2016), which are among the popular models for VRPs, consist of a master problem (MP) and several subproblems. Routes are searched in the subproblems and treated as variables in the MP, which is a concise set-covering or set-partitioning model. In this process, the substructure within the routes is omitted from the MP and only solved in the subproblems. Linear relaxed MP can be solved using classical column generation (Lübbecke and Desrosiers, 2005). Then branch-and-price algorithms can be developed by employing a branch and bound (B&B) framework. This approach has made considerable progresses in solving VRPs. However, such a concise MP can hardly be formulated for the LDDP due to the parking constraints must be maintained in the MP.

An alternative approach involves combining a trip-based model with a time-indexed formulation technique (Van den Akker et al., 2000; Avella et al., 2017), which is commonly used to model machine scheduling problems. The time scheduling horizon can be discretized into a set of small periods, and the parking constraints (i.e., lockers serving drones one by one) are enforced by limiting that a locker serves at most one drone in each period. Such a formulation is provided in the Supplementary materials. The main drawback of this approach is its intractable scale, as it often requires a large number of constraints and variables due to the extensive number of periods.

We also model the LDDP using a trip-based two-stage formulation, which is detailed in the Supplementary materials. In the first stage, only the routing problem is addressed and parking constraints are temporarily neglected. Then the parking sub-problem is solved in the second stage using the solution of the first stage. However, both models in the two stages are mixed integer programs. Solving the TTSP is challenging as a result.

For comparison, the well-known TSP formulation proposed by Dantzig et al. (1954) exhibits a simple but elegant structure. The core constraint in the formulation is to limit vertex degrees, which is achieved by selecting edges to connect the vertices. By dynamically generating sub-tour cuts (SEC) to strengthen the formulation, TSP can be solved in an iterative manner. This leads us to explore whether the LDDP has similar problem structures using digraph methods (Vogiatzis and Walteros, 2018; Vogiatzis and Camur, 2019). As a result, we find the sufficient and necessary conditions of feasible solutions. As a result, we transformed the LDDP into a tractable problem that can be modeled using structure-compressed formulations.

4. Sufficient and necessary conditions for feasible solutions

We start this section by introducing some preliminary definitions. Recall that c_v is the number of lockers deployed at site $v \in V$. Let s_v denote the number of drones and $r_v = c_v - s_v$ denote the number of idle parking platforms at site $v \in V$ before the delivery. We define the set $S = \{s_v | v \in V\}$ as the initial **state** of the delivery system. For description convenience, we define the state of site v as **active** if there is at least one drone at v . Similarly, the state of a site-set is defined as active if it contains an active site.

Now we introduce a digraph construction method for a given solution of the LDDP. Let $\mathcal{X}$ be a solution which may be feasible or not. Without loss of generality, we assume the flight legs of $\mathcal{X}$ contain all the loaded flight legs and some empty flight legs. A digraph $G_{\mathcal{X}} = (V, E_{\mathcal{X}})$ is constructed for $\mathcal{X}$ by the following two steps. First, a vertex is created for each site, and hence, the site

set V is reused to represent the set of vertices. A vertex is also called active if its corresponding site is active. The same calling applies to a subgraph that contains at least one active vertex. Second, a directed edge is created for each flight leg of $\mathcal{X}$. The created directed edge's tail and head vertex correspond to its associated flight leg's takeoff and landing site, respectively. The notation $E_{\mathcal{X}}$ represents the set of created edges. Please note that the underlying graph of $G_{\mathcal{X}}$ may not be connected.

In the constructed digraph, we use $d_v^-(\mathcal{X})$ and $d_v^+(\mathcal{X})$ to denote the out- and in-degrees of vertex $v \in V$, respectively. Then, the set $D_{\mathcal{X}} = \{d_v(\mathcal{X}) = d_v^-(\mathcal{X}) - d_v^+(\mathcal{X}) | v \in V\}$ is defined as the **degree difference** of $G_{\mathcal{X}}$. Given the above definitions and notations, we propose the sufficient and necessary conditions for feasible solutions for the LDDP.

Theorem 1. *Solution $\mathcal{X}$ is feasible if and only if its constructed digraph $G_{\mathcal{X}}$ satisfy the following two conditions: (i) $G_{\mathcal{X}}$ can be partitioned into a set of subgraphs, each of which has a connected underlying graph and an active initial state; (ii) $G_{\mathcal{X}}$ satisfies inequalities (1) and (2),*

$$r_v \geq -d_v(\mathcal{X}), v \in V, \quad (1)$$

$$s_v \geq d_v(\mathcal{X}), v \in V. \quad (2)$$

The necessity proof of Theorem 1 is relatively straightforward, while the sufficiency proof is more complex. Therefore, in the following, we will first prove the necessity.

Proof (Necessity of Theorem 1). Given that $\mathcal{X}$ is feasible, we need to derive that the constructed digraph $G_{\mathcal{X}}$ must satisfy the two conditions.

Condition (i) is straightforward. We partition $G_{\mathcal{X}}$ into a set of separate subgraphs, ensuring that the underlying graphs of any two subgraphs are not connected. Then the vertex set of each subgraph must be active. Otherwise, there must be a set of flight legs which cannot be executed due to the lack of drones, which contradicts the feasibility of $\mathcal{X}$.

Condition (ii) can be proved by deduction. Since $\mathcal{X}$ is feasible, all its flight legs can be executed. We denoted the ultimate state of delivery system by $S' = \{s'_v | v \in V\}$, where s'_v represents the number of drones at site v after all the flight legs get executed. Clearly, executing an outgoing flight leg will reduce the number of drones at a site, while executing an incoming flight leg will increase it. Therefore, the equation $s'_v = s_v + d_v^+(\mathcal{X}) - d_v^-(\mathcal{X}) = s_v - d_v(\mathcal{X})$ holds. As we can easily analyze, it must hold that $s'_v \leq c_v$, as otherwise there would not be enough parking platforms for the drones in the end. Consequently, we have $s_v - d_v(\mathcal{X}) \leq c_v$, which simplifies to $r_v = c_v - s_v \geq -d_v(\mathcal{X})$. Similarly, it must hold that $s'_v \geq 0$, as there cannot be a negative number of drones. Therefore, we also have $s_v - d_v(\mathcal{X}) \geq 0$, which implies $s_v \geq d_v(\mathcal{X})$. $\square$

In the following, we will prove the sufficiency of Theorem 1. Our proof follows the primary methodology of partitioning the digraph into paths and cycles. To complete the proof, we need to introduce three lemmas. For the sake of clarity, we define a subgraph as **executable** if a feasible drone transportation plan can be made for all of its flight legs. Please note that when discussing the executability of a subgraph, we only consider the subgraph itself.

Lemma 1. *Let subgraph $p_{\mathcal{X}}$ be a path of $G_{\mathcal{X}}$ with tail vertex $\tau_{\mathcal{X}}^p$ and head vertex $\sigma_{\mathcal{X}}^p$. If a state satisfies that site $\tau_{\mathcal{X}}^p$ is active and there is one idle parking platform at site $\sigma_{\mathcal{X}}^p$, path $p_{\mathcal{X}}$ is executable. Furthermore, when the flight legs execution of $p_{\mathcal{X}}$ is finished, both the state and degree difference of the tail (resp. head) vertex decrease (resp. increase) by 1. The state and the degree difference of other vertices in path $p_{\mathcal{X}}$ remain unchanged.*

Proof. Without loss of generality, we suppose it is the initial state which satisfies the conditions of Lemma 1. The path $p_{\mathcal{X}}$ is denoted by a vertex permutation $\langle v_0 = \tau_{\mathcal{X}}^p, v_1, \dots, v_n = \sigma_{\mathcal{X}}^p \rangle$. An edge $a_k = \langle v_k, v_{k+1} \rangle \subseteq p_{\mathcal{X}}$ must exist so that there is at least one drone at site v_k and one idle parking platform at site v_{k+1} . (This can be proved by contradiction.) Therefore, the flight leg represented by a_k can be executed directly. When the execution of this flight leg is completed, at least one idle parking platform emerges at site v_k and one drone parks at site v_{k+1} . Hence, an updated state is obtained. The edge a_k is removed from $p_{\mathcal{X}}$ since the corresponding flight leg gets executed.

The path $p_{\mathcal{X}}$ is then partitioned into two sub-paths: $p_{\mathcal{X}}^1 = \langle v_0 = \tau_{\mathcal{X}}^p, v_1, \dots, v_k \rangle$ and $p_{\mathcal{X}}^2 = \langle v_{k+1}, \dots, v_{n-1}, v_n = \sigma_{\mathcal{X}}^p \rangle$. Either $p_{\mathcal{X}}^1$ or $p_{\mathcal{X}}^2$ satisfies the condition of Lemma 1 with the updated state. Thus, such an operation can also be conducted within $p_{\mathcal{X}}^1$ and $p_{\mathcal{X}}^2$. Consequently, a recursive use of this operation can be performed so that all the flight legs get executed. The changes of the state and the degree difference after the execution can be derived straightforwardly. $\square$

Lemma 2. *Let $o_{\mathcal{X}} = \{V_{\mathcal{X}}^o \subseteq V_{\mathcal{X}}, E_{\mathcal{X}}^o \subseteq E_{\mathcal{X}}\}$ be a cycle of $G_{\mathcal{X}}$. If there is a vertex $v \in V_{\mathcal{X}}^o$ with an active state, the cycle $o_{\mathcal{X}}$ is executable. Furthermore, the states and the degree differences of vertices in $V_{\mathcal{X}}^o$ remain unchanged when the execution of flight legs in $o_{\mathcal{X}}$ is finished.*

Proof. If there is an idle parking platform at one of the sites in $V_{\mathcal{X}}^o$, then $o_{\mathcal{X}}$ can be treated as a special path that satisfies the conditions of Lemma 1. Therefore, $o_{\mathcal{X}}$ is executable. Otherwise, if there are no idle parking platforms, the flight legs can be executed synchronously. Namely, $|V_{\mathcal{X}}^o|$ drones are selected to execute the $|E_{\mathcal{X}}^o| = |V_{\mathcal{X}}^o|$ flight legs, and all the drones take off simultaneously. There would be no parking conflicts, and the execution is practical. Hence, $o_{\mathcal{X}}$ is executable. The changes in the state and the degree difference after the execution of a cycle's flight legs can be derived straightforwardly. $\square$

Lemma 3. *Let $Q_{\mathcal{X}} \subseteq G_{\mathcal{X}}$ be a connected subgraph that is executable. If digraph $G_{\mathcal{X}}$ with a state follows inequalities (1) and (2), then the residual digraph $G'_{\mathcal{X}} = G_{\mathcal{X}} - Q_{\mathcal{X}}$ with the updated state obtained by executing $Q_{\mathcal{X}}$ must follow the inequalities as well.*

Proof. If $Q_{\mathcal{X}}$ is a cycle, both the state and the degree difference will not change after the executing (Lemma 2). Therefore, the lemma holds. If $Q_{\mathcal{X}}$ is a path, only the states and degree differences of the two end points will change after execution. However, in this change, the state and degree difference of the tail (or head) vertex will decrease (or increase) by 1. When adding or subtracting a number to both sides of the inequalities, they still hold. Therefore, the lemma also holds. $\square$

Lemma 1–2 reveal that which type of paths and cycles are executable. Lemma 3 show a critical feature of the constructed digraph. The three lemmas will be used in the sufficiency proof of Theorem 1.

Proof (Sufficiency of Theorem 1). Given that the constructed digraph $G_{\mathcal{X}}$ satisfies condition (i) and (ii), we need to derive that $\mathcal{X}$ is a feasible solution.

Condition (i) ensures that the underlying graph of each partitioned subgraph is connected. Consequently, $G_{\mathcal{X}}$ is executable if every subgraph is independently executable. Therefore, we only need to prove that any a subgraph $Q_{\mathcal{X}} = \{V_{\mathcal{X}}^Q, E_{\mathcal{X}}^Q\}$ of $G_{\mathcal{X}}$ is executable.

According to graph theory (West et al., 2001), the edges of $E_{\mathcal{X}}^Q$ can be partitioned into two sets: $O_{\mathcal{X}}^Q$ comprising cycles and $P_{\mathcal{X}}^Q$ comprising paths.

Let $p_{\mathcal{X}} \in P_{\mathcal{X}}^Q$ be a path. The state of its tail vertex $\tau_{\mathcal{X}}^p$ follows that $s_v \geq d_v(\mathcal{X}) > 0, v = \tau_{\mathcal{X}}^p$. Notice that the first part $s_v \geq d_v(\mathcal{X})$ is obtained according to condition (ii) that the initial state satisfies inequalities (1) and (2). The state of its head vertex $\sigma_{\mathcal{X}}^p$ satisfies $r_v \geq -d_v(\mathcal{X}) > 0, v = \sigma_{\mathcal{X}}^p$. Therefore, $p_{\mathcal{X}}$ is executable (Lemma 1). When the execution of flight legs in $p_{\mathcal{X}}$ is finished, the updated state and the degree differences still satisfy inequalities (1) and (2) (Lemma 3). Therefore, this operation can be repeated so that the flight legs of all the paths get executed.

Please note that the underlying graph of $Q_{\mathcal{X}}$ is connected. Hence, at least one vertex of the cycles in $O_{\mathcal{X}}^Q$ will be state-active during the executing of the flight legs of $P_{\mathcal{X}}^Q$. Once a vertex of a cycle in $O_{\mathcal{X}}^Q$ becomes state-active, the flight legs of the cycle can be executed (Lemma 2). The state and the degree differences still satisfy inequalities (1) and (2) when the execution is finished (Lemma 3). By this method, the flight legs of all cycles in $O_{\mathcal{X}}^Q$ can be executed. $\square$

Theorem 1 relates to the b -transshipments theorem (Schrijver et al., 2003) in which a notion of excess is provided. More specifically, the excess of vertex $v \in V$ is defined as $excess_f(v) := f(d_v^-(\mathcal{X})) - f(d_v^+(\mathcal{X}))$ where f is an edge function: $E_{\mathcal{X}} \rightarrow \mathbb{R}$, e.g., a flow on the digraph. If equation $excess_f(v) = b_v$ holds for $v \in V$ where b_v is an entry of vector $b \in \mathbb{R}^{|V|}$, function f is defined as a b -transshipments. Therefore, the degree difference defined in this paper is a special case of the excess. However, Theorem 1 aims at answering the question that what excess values of a digraph will correspond to a feasible solution of the LDDP. The b -transshipments theorem primarily discuss the existence of edge functions so that the excess equals to some special values. Although related, their purposes are apparently different.

Theorem 1 is powerful for finding optimal solutions for the LDDP. For a given instance of the LDDP, an initial set of flight legs can be obtained by assigning a loaded flight leg for each delivery task. Then a transformed problem can be modeled to find the optimal empty flight legs such that the constructed digraph with the initial state satisfies the conditions of Theorem 1. The objective of the transformed problem is to minimize the total distance of added empty flight legs. Once the optimal empty flight legs are found, practical schedules can be computed following the main steps in the proof of Theorem 1. This motivates us to develop a two-stage solution method, which will introduced in the following section.

5. A two-stage solution method

Our solution method consists of two stages. In the first stage, a B&C algorithm is developed to solve the transformed problem. The flight legs obtained from this stage serve as input for the second stage. In the second stage, a heuristic is employed to construct a practical schedule for the LDDP. The following text first introduces the formulations for the transformed problem, then presents the B&C algorithm, and finally elaborates on the heuristic.

5.1. Formulations for transformed problem

The transformed problem aims to find the optimal empty flight legs, such that the constructed digraph with the initial state satisfies the conditions of Theorem 1. Let $e \in \mathcal{E}$ be an empty flight leg, where $\mathcal{E}$ is the set of all candidate empty flight legs. The travel distance of e is denoted by λ_e . A nonnegative integer variable $x_e \in \mathbb{N}$ is used to represent the selection times of e . Then the objective of the transformed problem can be rewritten as:

$$\min z = \sum_{e \in \mathcal{E}} \lambda_e x_e \quad (3)$$

Condition (ii) of Theorem 1 reveals the relationship between the initial state and the degree difference of a feasible solution. It can be analyzed that integral linear constraints can easily be created according to this condition. Let μ_e^v be a binary parameter that denotes the relationship between flight leg $e \in \mathcal{E}$ and site $v \in V$. Specifically, $\mu_e^v = 1$ if e corresponds to an incoming flight leg for v ; $\mu_e^v = -1$ if e corresponds to an outgoing flight leg for v ; and $\mu_e^v = 0$ otherwise. Let $d_v(\mathcal{Y})$ be the degree difference of vertex $v \in V$ in digraph $G_{\mathcal{Y}}$ where $\mathcal{Y}$ is the initial set of flight legs created by assigning a loaded flight leg for each delivery task $t \in T$. The total degree difference of v after selecting x_e times of e is given by $d_v = d_v(\mathcal{Y}) + \sum_{e \in \mathcal{E}} x_e \mu_e^v$. The constraints corresponding to condition (ii) can be written as follows:

$$\sum_{e \in \mathcal{E}} x_e \mu_e^v \geq -r_v - d_v(\mathcal{Y}), v \in V \quad (4)$$

$$\sum_{e \in \mathcal{E}} x_e \mu_e^v \leq s_v - d_v(\mathcal{Y}), v \in V \quad (5)$$

$$x_e \in \mathbb{N}, e \in \mathcal{E} \quad (6)$$

Constraints (4) and (5) are obtained according to inequalities (1) and (2). As can be easily analyzed, objective (3) together with constraints (4) – (6) constitute an LB formulation. For the sake of descriptive convenience, this program is represented as **F1**.

F1

Objective: (3),

Subject to: (4)–(6).

The size of the coefficient matrix of **F1** is relatively small. Each column corresponds to a flight leg of $\mathcal{E}$. Hence, the number of all columns is equal to $|\mathcal{E}|$. The number of all rows is equal to $2|V|$. The coefficient matrix is a row-doubled incidence matrix. There are exactly four nonzero entries in each column. In addition to the tractable scale, the coefficient matrix possesses a merit of total unimodularity, as demonstrated in [Proposition 1](#).

Proposition 1. *The coefficient matrix of **F1** is totally unimodular.*

Proof. Please see the Supplementary materials.

There is a fundamental theory in integer programming which states that the polyhedron represented by the linear-relaxed program will be integral if and only if the coefficient matrix of the corresponding integer program is totally unimodular ([Conforti et al., 2014](#)). Hence, an integral solution must be obtained by solving the linear relaxed program of **F1**. Please note that although it is integral, this solution may violate condition (i) of [Theorem 1](#). In the following, we will address this issue by formulating a UB model.

For a given instance of the LDDP, a digraph $G_{\mathcal{Y}}$ can be constructed based on the initial set of flight legs $\mathcal{Y}$. Clearly, the underlying graph of $G_{\mathcal{Y}}$ may not be connected. Recall that condition (i) of [Theorem 1](#) requires the constructed digraph can be partitioned into a set of subgraphs with connected underlying-graph and active vertices. In order to satisfy this condition, a natural approach is to partition $G_{\mathcal{Y}}$ into subgraphs and use empty flight legs to connect them. As can be easily derived, the number of used empty flight legs should be as small as possible to achieve the objective. Therefore, we need to minimize the number of subgraphs that partitioned from $G_{\mathcal{Y}}$. So we partition $G_{\mathcal{Y}}$ into a set of separate subgraphs. Here, ‘separate’ means that the underlying graphs of any two subgraphs are not connected.

A separate subgraph is either active or inactive. Let $\mathcal{G}^{ac}$ and $\mathcal{G}^{in}$ denote the set of active and inactive subgraphs, respectively. Then we impose that at least one flight leg is added to connect the subgraphs of $\mathcal{G}^{ac}$ and each subgraph of $\mathcal{G}^{in}$. Let ϕ be the subgraph index, and V_{ϕ} the vertex set of ϕ . Let $e_{u,v}$ denote the flight leg which originates from site u and lead to v . A set of flight legs for subgraph $\phi \in \mathcal{G}^{in}$ is represented by $\Theta_{\phi} = \{e_{u,v} \in \mathcal{E} | u \in \cup_{\phi' \in \mathcal{G}^{ac}} V_{\phi'}, v \in V_{\phi}\}$. With those notations, the impositions can be represented as constraints (7).

$$\sum_{e \in \Theta_{\phi}} x_e \geq 1, \phi \in \mathcal{G}^{in} \quad (7)$$

Adding constraints (7) to **F1** yields a new formulation which is referred to as **F2**. Evidently, a feasible solution for the transformed problem can always be obtained by solving **F2**. Hence, **F2** is a UB formulation.

F2

Objective: (3),

Subject to: (4)–(7).

The number of rows in Constraints (7) is equal to $|\mathcal{G}^{in}|$, which is fewer than the total number of vertices. Therefore, formulation **F2** is also very compact. Moreover, the coefficient matrix of Constraints (7) is also totally unimodular, as demonstrated in [Proposition 2](#).

Proposition 2. *The coefficient matrix of Constraints (7) is totally unimodular.*

Proof. Please see the Supplementary materials.

The coefficient matrix of **F2** consists of two submatrices, each of which is totally unimodular. Integral solutions are very likely to be computed. Numerical tests show that **F2** can be efficiently solved. In the next subsection, we will discuss narrowing the gap between **F1** and **F2**.

5.2. Branch-and-cut for transformed problem

Note that solving **F2** we may obtain sub-optimal solutions for the transformed problem. [Fig. 2](#) shows an example. We use a solid dot to represent an active vertex and a hollow dot to represents an inactive vertex. In [Fig. 2\(a\)](#), three inactive subgraphs are connected by three flight legs that direct from an active subgraph. [Fig. 2\(b\)](#) provides a contrastive case where the three are connected by a path instead. If the total distance of the three flight legs is larger than that of the path, the solution displayed in [2\(a\)](#) is inferior than that in [Fig. 2\(b\)](#). Therefore, solving **F2** we may obtain sub-optimal solutions.

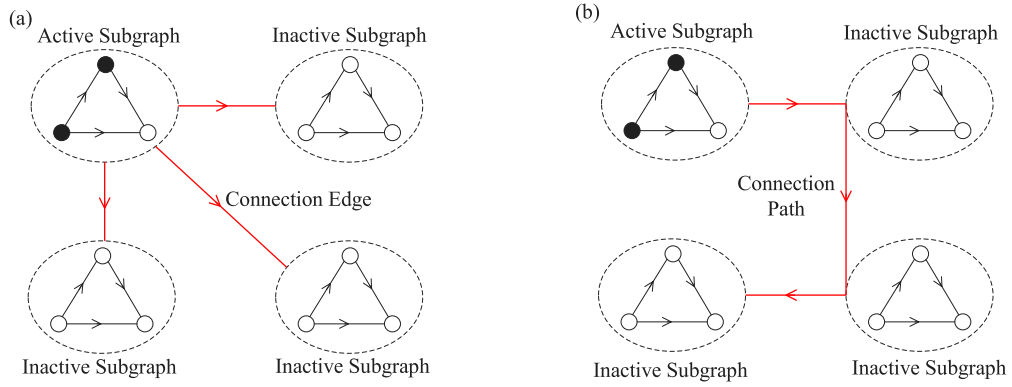


Fig. 2. A comparison between connecting flight legs and connecting path.

Since constraints (7) are too strong, we put forward a relaxed version. The notation $\bar{V}_\phi = \{V \setminus V_\phi\}$ represents the remaining vertices for subgraph ϕ . The set $\Omega_\phi = \{e_{u,v} \in \mathcal{E} | u \in \bar{V}_\phi, v \in V_\phi\}$ contains the flight legs originating from the sites of $\bar{V}_\phi$ and leading to the sites of V_ϕ . The new constraints require the addition of at least one such empty flight leg for each inactive subgraph, as shown in (8).

$$\sum_{e \in \Omega_\phi} x_e \geq 1, \phi \in \mathcal{G}^{\text{in}} \quad (8)$$

Replacing constraints (7) with (8) in F2 can prevent the excluding of optimal solutions. However, feasibility is not ensured by imposing constraints (8) only. For example, the added empty flight legs may connect multiple inactive subgraphs.

If such a situation emerges, inactive subgraphs may still exist. In this case, we can update the set $\mathcal{G}^{\text{in}}$ to include any newly identified inactive subgraphs. Then an updated version of constraints (8) can be identified and added. This operation can be repeated. As such, constraints (8) can be treated as a type of cut plane for the transformed problem. We refer to them as PC cuts.

When a set of PC cuts is added, the resulting coefficient matrix may or may not be totally unimodular. Consequently, the solution of the linear relaxation may or may not be integral. If a fractional solution is obtained, it is difficult to identify any PC cuts. This suggests the employment of the B&B framework and the development of a B&C algorithm.

The B&B search begins by exploring the root node and continues by expanding branches. At each node of the tree, an F1's linear relaxed program (LRP) is solved. If the solution is integral, a check for satisfying the conditions is conducted. If the solution satisfy the conditions, the node is fathomed. Otherwise, there must be at least one inactive subgraph. Then new PC cuts can be identified and added, followed by solving the node's new LRP once again. This process is repeated until the solution satisfy the conditions or the solution is fractional. If a fractional solution is obtained, a branching operation is performed.

Three searching strategies are combinedly used to find the next branching node. They are the minimum LB first, the maximum LB first, and the breadth-first strategies. One of them is used initially and replaced by another if the LB shows little improvement after a number of node explorations, which facilitates a more robust search. F2 is the UB formulation and hence, its optimal objective value can be used to prune branches. As the B&B framework implicitly enumerates possible values for variables, the proposed B&C is an exact algorithm.

It is worth noting that the PC cuts identified at one node can be used at any other nodes. In addition, the most important merit of this approach may be the concise structure of F1. Although the linear relaxation of F1 with added PC cuts may no longer guarantee integrality, integral solutions are still very likely to be produced. Consequently, branching operations may rarely be needed. As a result, a small-scale B&B tree may suffice, making the algorithm highly efficient.

5.3. Heuristic for practical schedules

The optimal empty flight legs will be obtained by solving the transformed problem in the first stage of our method. These empty flight legs together with the loaded flight legs, referred to as optimal flight legs, serve as the input for the second stage. According to Theorem 1, a feasible solution for the LDDP must exist in terms of that input. Hence, we develop a heuristic to construct a practical schedule for the LDDP.

Our heuristic is developed inspired by the proof process of Theorem 1. Hence, firstly, we construct a digraph according to the input flight legs. Secondly, we partition the digraph into three sets of subgraphs, namely, a set of paths P , a set of active cycles O_A , and a set of inactive cycles O_I . At last, we make schedules for executing the corresponding flight legs. The details of the heuristic is shown in Algorithm 1. Line 3 to 14 show the digraph-partition procedure. Line 15 to 25 show the schedule-making procedure. When executing a flight leg, a drone is first selected, and then the times for takeoff, travel, and landing operations are determined. The output of the heuristic is a practical schedule stating which drone executes which flight legs in which time.

Theorem 1 ensures that at least one practical schedule can be constructed in terms of those flight legs. The main steps of the heuristic follow the proof procedure of Theorem 1. Therefore, one practical schedule can be obtained and no extra flight legs are

Algorithm 1: A heuristic for practical schedule

Input: the optimal flight legs
Output: a feasible solution for the LDDP

```

1  construct digraph  $G = \{V, E\}$  according to the flight legs;
2  initialize subgraph sets:  $P \leftarrow \emptyset, O_A \leftarrow \emptyset, O_I \leftarrow \emptyset$ ;
3  while  $\exists v \in V$  s.t.  $d_v \neq 0$  do
4      find path  $p \subseteq G$  s.t.  $d_u > 0, d_w < 0$ ;
5      //  $u$  and  $w$  are tail and head vertices of  $p$ , respectively;
6       $P \leftarrow \{P, p\}$ ;
7      remove the edges of  $p$  from  $E$ ;
8  while  $E \neq \emptyset$  do
9      find cycle  $o \subseteq G$ ;
10     if  $o$  is active then
11          $O_A \leftarrow \{O_A, o\}$ ;
12     else
13          $O_I \leftarrow \{O_I, o\}$ ;
14     remove the edges of  $o$  from  $E$ ;
15 for  $o$  in  $O_A$  do
16     execute the flight legs of  $o$ ;
17     // the state is updated simultaneously, the same below;
18     remove  $o$  from  $O_A$ ;
19 for  $p$  in  $P$  do
20     while the edges set of  $p$  is nonempty do
21         find an executable edge  $e \in p$ ;
22         execute the flight leg corresponding to  $e$ ;
23         if the head of  $e$  associates with  $o \in O_I$  then
24             execute the flight legs of  $o$ ;
25             remove  $o$  from  $O_I$ ;
26         remove  $e$  from  $p$ ;
```

added in the constructing process. Therefore, the total distance traveled by the drones will not increase and it equals to the total distance of these flight legs. The output of the heuristic is an optimal solution for the LDDP.

The heuristic is very efficient. The most time-consuming step is finding a path or a cycle from the digraph. However, the worst-case of this step is a complete traversal of all vertices. Hence, the number of all these explorations is less than $|E||V|$. Our numerical tests suggest that the heuristic is fast enough for several highly simulated applications. In the next section, we will introduce the computational experiments.

6. Computational experiments

The proposed two-stage solution method has been verified through a series of numerical tests conducted with both simulated and randomly generated instances. The algorithms are implemented in the Java programming language and run on a personal computer equipped with an i7-1165G7 CPU, 16 GB of RAM memory, and the Windows 11 Home Basic operating system. The ILOG Cplex software is used as the LP solver in the program. The feasibility of obtained practical schedules is verified for each instance.

6.1. Tests on simulated instances

The effectiveness of the algorithm for solving real-world problems is verified using simulated instances. Nine Chinese cities have been selected as the test regions, some of which already operate an LDD business or plan to do so in the near future. Residential areas, hospitals, restaurants, and commercial centers in each city are selected to be alternative sites. The location data (longitude and latitude) for each site and distance data are calculated using a map software. The tests adopt the flying velocity of Antwork RA3 drone, which is already in use in Hangzhou city. The loading or unloading time is set to 2 min, i.e., 120 s.

We construct 45 small-scale instances to verify our method's correctness and efficiency. Each instance includes 5 sites and no more than 40 tasks. The task data are randomly generated following a uniform distribution. For comparison, we also test these instances using the introduced two formulations, namely, the TTIF and the TTSF. Please note that due to the intractability of parking constraints (E5) in the TTIF, we have relaxed these constraints. Otherwise, solving the TTIF would be impossible. Consequently, the obtained solutions may not be feasible for the LDDP. The relaxed TTIF and the first stage program of the TTSF are solved using a branch and price algorithm (Desaulniers et al., 2006). Solving some instances is very time consuming; hence, we employ a time-out termination condition. Specifically, if the CPU time of a solving process exceeds 600 s, it will be forcibly aborted. The statistical results are listed in Table 1.

As can be observed from Table 1, the two comparative methods compute optimal solutions for some instances. The objective values of these solutions are consistent with those of the two-stage method. This provides empirical evidence that the two-stage

Table 1
Statistical comparison results on small-scale instance.

Instance	City	#Task	Relaxed TTTF				TTSF				Two-stage method	
			Feasible	Optimal	Obj	CPU(s)	Feasible	Optimal	Obj	CPU(s)	Obj	CPU(s)
L1	Beijing	10	295 627	0.18	✗	✗	295 627	0.50	✓	✓	295 627	0.09
L2		15	337 056	0.75	✗	✗	337 056	0.90	✓	✓	337 056	0.08
L3		20	402 902	1.61	✗	✗	402 902	3.76	✓	✓	402 902	0.09
L4		30	N/A	600.00	N/A	N/A	N/A	600.00	N/A	N/A	610 496	0.08
L5		40	N/A	600.00	N/A	N/A	N/A	600.00	N/A	N/A	877 759	0.09
L6	Shanghai	10	173 463	0.13	✗	✗	173 463	0.22	✓	✓	173 463	0.08
L7		15	360 161	1.22	✗	✗	360 161	2.51	✓	✓	360 161	0.09
L8		20	N/A	600.00	N/A	N/A	N/A	600.00	N/A	N/A	407 343	0.09
L9	Nanjing	30	636 653	10.10	✓	✓	636 653	20.06	✓	✓	636 653	0.09
L10		40	N/A	600.00	N/A	N/A	N/A	600.00	N/A	N/A	870 477	0.09
L11		10	135 849	0.09	✗	✗	135 849	0.12	✓	✓	135 849	0.09
L12		15	204 966	0.22	✗	✗	204 966	0.35	✓	✓	204 966	0.08
L13		20	331 977	18.86	✓	✓	331 977	15.07	✓	✓	331 977	0.08
L14	Chengdu	30	N/A	600.00	N/A	N/A	N/A	600.00	N/A	N/A	397 776	0.06
L15		40	N/A	600.00	N/A	N/A	N/A	600.00	N/A	N/A	566 001	0.06
L16		10	140 670	0.28	✗	✗	140 670	0.36	✓	✓	140 670	0.06
L17		15	240 147	0.75	✗	✗	240 147	0.84	✓	✓	240 147	0.06
L18		20	N/A	600.00	N/A	N/A	N/A	600.00	N/A	N/A	290 544	0.06
L19	Wuhan	30	N/A	600.00	N/A	N/A	N/A	600.00	N/A	N/A	462 137	0.07
L20		40	N/A	600.00	N/A	N/A	N/A	600.00	N/A	N/A	526 939	0.08
L21		10	231 519	0.10	✓	✓	231 519	0.14	✓	✓	231 519	0.07
L22		15	355 330	59.68	✓	✓	355 330	53.63	✓	✓	355 330	0.07
L23		20	N/A	600.00	N/A	N/A	N/A	600.00	N/A	N/A	636 901	0.07
L24	Xian	30	N/A	600.00	N/A	N/A	N/A	600.00	N/A	N/A	677 234	0.07
L25		40	N/A	600.00	N/A	N/A	N/A	600.00	N/A	N/A	873 924	0.08
L26		10	172 149	0.18	✗	✗	172 149	0.24	✓	✓	172 149	0.10
L27		15	234 391	0.26	✗	✗	234 391	0.53	✓	✓	234 391	0.09
L28		20	376 586	2.29	✓	✓	376 586	2.40	✓	✓	376 586	0.08
L29	Hefei	30	N/A	600.00	N/A	N/A	N/A	600.00	N/A	N/A	462 888	0.09
L30		40	N/A	600.00	N/A	N/A	N/A	600.00	N/A	N/A	683 759	0.08
L31		10	158 086	0.16	✗	✗	158 086	0.22	✓	✓	158 086	0.07
L32		15	251 288	0.20	✗	✗	251 288	0.30	✓	✓	251 288	0.07
L33		20	N/A	600.00	N/A	N/A	N/A	600.00	N/A	N/A	315 389	0.07
L34	Hangzhou	30	N/A	600.00	N/A	N/A	N/A	600.00	N/A	N/A	507 982	0.07
L35		40	N/A	600.00	N/A	N/A	N/A	600.00	N/A	N/A	640 478	0.07
L36		10	200 665	0.27	✗	✗	200 665	0.56	✓	✓	200 665	0.07
L37		15	213 825	0.39	✗	✗	213 825	0.69	✓	✓	213 825	0.07
L38		20	265 080	3.37	✗	✗	265 080	9.98	✓	✓	265 080	0.09
L39	Shenzhen	30	N/A	600.00	N/A	N/A	N/A	600.00	N/A	N/A	415 648	0.09
L40		40	N/A	600.00	N/A	N/A	N/A	600.00	N/A	N/A	599 495	0.07
L41		10	171 108	0.14	✓	✓	171 108	0.19	✓	✓	171 108	0.07
L42		15	216 532	19.83	✗	✗	216 532	17.51	✓	✓	216 532	0.08
L43		20	N/A	600.00	N/A	N/A	N/A	600.00	N/A	N/A	370 810	0.07
L44		30	N/A	600.00	N/A	N/A	N/A	600.00	N/A	N/A	418 600	0.07
L45		40	N/A	600.00	N/A	N/A	N/A	600.00	N/A	N/A	593 765	0.06
Mean				296.02				296.25				0.08

The symbol ‘#’ denotes ‘the number of’. Column ‘#Task’ indicates the number of tasks; Column ‘Obj’ represents the objective value of the LDDP; Column ‘CPU(s)’ lists the used CPU time; Column ‘Feasible’ denotes whether the computed solution is feasible or not; The abbreviation ‘N/A’ stands for ‘not applicable’, which indicates that the result could not be collected as time or memory has run out.

method computes the optimal solution. In terms of CPU time, the two-stage method uses about 0.1 s to compute an optimal solution for each instance. As comparison, the other two methods need more CPU time to compute comparable solutions. For some instances with more than 30 tasks, the other two methods fail to find any solutions. This highlights the efficiency strength of the two-stage method.

To simulate the real applications, we generate three sets of instances. SET 1 includes instances generated to simulate a catering express scenario in which parcels are always carried from 3–5 restaurants to 20 residential sites (“Less to More”). SET 2 involves instances generated to simulate a sample submission scenario, in which parcels are always carried from 20 residential sites to 3–5 hospitals (“More to Less”). Instances in SET 3 are generated to simulate an instant express scenario in which the delivery tasks are between residential sites and commercial centers (“More to More”). The results show that each simulated instance is solved within one second. Statistical results for all 81 instances are listed in Table 2.

The optimal solutions are computed very efficiently since all the CPU times are less than one second. The distance ratio of empty to loaded flight legs is calculated and shown in the ‘Ratio’ column. This number indicates the degree of necessary travel distance resulting from empty flight legs in the optimal transportation plan. On average, an additional distance of 72% and 48% is required for SETs 1 and 2, respectively. In contrast, this number is only 12% in SET 3. In other words, significantly less LDDP is needed

Table 2
Statistical results for simulated instances.

City	#Task	SET 1: Less to more				SET 2: More to less				SET 3: More to more			
		ITC	T_dist	E_dist	Ratio	ITC	T_dist	E_dist	Ratio	ITC	T_dist	E_dist	Ratio
Beijing	20	S1	72 945	25 391	35%	S28	175 320	66 990	38%	S55	199 767	27 019	14%
	50	S2	217 056	196 303	90%	S29	438 253	228 058	52%	S56	597 219	82 698	14%
	80	S3	368 076	352 875	96%	S30	783 666	401 913	51%	S57	956 244	185 817	19%
Shanghai	20	S4	109 736	40 852	37%	S31	112 865	62 378	55%	S58	149 959	7723	5%
	50	S5	285 617	172 548	60%	S32	248 034	106 369	43%	S59	372 259	29 810	8%
	80	S6	594 421	514 070	86%	S33	415 906	228 011	55%	S60	536 804	58 340	11%
Nanjing	20	S7	96 234	54 469	57%	S34	267 712	114 860	43%	S61	197 273	17 288	9%
	50	S8	170 067	138 103	81%	S35	455 908	195 754	43%	S62	440 167	49 355	11%
	80	S9	415 197	265 189	64%	S36	662 222	310 718	47%	S63	910 930	120 575	13%
Chengdu	20	S10	126 189	99 998	79%	S37	98 998	28 587	29%	S64	198 646	21 940	11%
	50	S11	222 532	126 146	57%	S38	246 500	110 222	45%	S65	408 210	28 198	7%
	80	S12	344 046	181 558	53%	S39	389 590	170 317	44%	S66	640 866	77 641	12%
Wuhan	20	S13	95 377	73 588	77%	S40	217 607	78 370	36%	S67	203 174	11 346	6%
	50	S14	171 098	161 888	95%	S41	671 517	316 510	47%	S68	557 261	92 507	17%
	80	S15	247 018	253 794	103%	S42	1 016 559	471 370	46%	S69	902 433	169 227	19%
Xian	20	S16	241 483	113 195	47%	S43	99 076	37 382	38%	S70	235 622	36 643	16%
	50	S17	746 675	282 290	38%	S44	218 650	107 977	49%	S71	522 216	42 072	8%
	80	S18	1 147 381	723 898	63%	S45	322 265	199 870	62%	S72	815 840	81 756	10%
Hefei	20	S19	236 995	206 548	87%	S46	94 173	36 234	38%	S73	165 133	21 256	13%
	50	S20	425 405	364 791	86%	S47	258 396	107 260	42%	S74	417 940	70 847	17%
	80	S21	735 556	669 354	91%	S48	426 440	153 261	36%	S75	648 002	70 726	11%
Hangzhou	20	S22	85 580	51 980	61%	S49	156 640	44 119	28%	S76	152 369	21 746	14%
	50	S23	210 383	175 775	84%	S50	380 719	180 631	47%	S77	359 960	50 766	14%
	80	S24	337 980	304 219	90%	S51	644 798	452 150	70%	S78	662 997	89 253	13%
Shenzhen	20	S25	107 719	82 511	77%	S52	199 443	129 721	65%	S79	169 724	16 124	10%
	50	S26	263 846	194 000	74%	S53	412 645	312 005	76%	S80	375 575	46 948	13%
	80	S27	344 529	275 426	80%	S54	609 938	484 010	79%	S81	597 881	82 643	14%
Mean					72%				48%				12%

'ITC' is an abbreviation for 'Instance'; Terms 'T_dist' and 'E_dist' represent the distances of delivery and empty flight legs, respectively; The ratio is computed according to E_dist/T_dist .

to complete delivery tasks for the instances in SET 3, which may be explained by the characters of the three sets. For example, instances in SET 1 simulate a catering service scene, in which parcels are always transported from 3–5 restaurants to 20 residential sites. As such, the number of departure sites in SET 1 is much smaller. These sites then become conflicting centers that are very 'busy' during transportation. Making a feasible transportation plan requires a large number of empty flight legs to eliminate these conflicts. As a result, the ratios in SETs 1 and 2 are much larger.

Tests on simulated instances reveal that the additional travel distance required depends heavily on the type of delivery scenario. Based on this observation, we propose a strategy of positioning drones to reduce the additional costs. This may involve deploying more lockers at sites that are likely to experience congestion during day-to-day deliveries. Overall, the proposed two-stage method is promising for the LDDP in real-world environments.

6.2. Tests with random instances generated following a uniform distribution

Subsequently, tests are conducted on a set of thirty random instances, denoted as R1–R30. This set, labeled as SET 4, comprises ten groups, each consisting of three instances with the same number of sites. The smallest instance involves 100 sites and 50 tasks, while the largest instance includes 1000 sites and 1500 tasks. The coordinate location and the number of lockers at each site vary within integer ranges of $[0, 10000]$ and $[1, 4]$, respectively. All random data are generated following a uniform distribution. All 30 instances are solved to optimality in acceptable CPU time, typically taking less than 50 s. The statistical instance data and computation results are presented in Table 3.

The number of drones is nearly 50% of the number of lockers, indicating that nearly half the lockers are idle. This is because the number of lockers and drones are generated uniformly. Since the number of lockers takes values from the integer intervals $[1, 4]$ following a uniform distribution, the probability expectation value for the number of lockers at a given site is $(1 + 4)/2 = 2.5$. Furthermore, the presence of a drone on a locker is also randomly generated following a uniform distribution. Therefore, the probability expectation value for the number of drones at a given site is $(0 + 2.5)/2 = 1.25$. As a result, the total number of drones for all instances is nearly 1.25 times larger than the number of sites. This is evident in Table 3.

The overall CPU time increases with the number of sites and ranges from 1 s to 49 s, indicating the algorithm is highly efficient for solving these instances. The optimal solutions for 28 instances are obtained by solving model F1, suggesting that F1 is a very tight LB formulation. This is also evident from the number of cuts used in Table 3. Specifically, 28 out of 30 instances are solved to optimality without the need for additional cuts. The remaining two instances, namely R5 and R25, require the addition of one cut. These numbers also highlight the tightness of the formulations.

Table 3
General statistical results for random instances in SET 4.

ITC	#Site	#Task	T_dist	#Drone	D_ratio	Opt*	F1	F2	Init_gap	CPU(s)	#Cuts
R1	100	50	217 851	126	50%	19 947	19 947	27 507	27%	1	0
R2	100	100	445 308	127	50%	57 433	57 433	58 910	3%	1	0
R3	100	150	667 004	138	57%	105 050	105 050	105 050	0%	1	0
R4	200	100	399 164	252	49%	47 630	47 630	56 944	16%	1	0
R5	200	200	785 117	231	48%	97 046	96 876	101 108	4%	1	1
R6	200	300	1 270 745	257	53%	145 258	145 258	147 736	2%	1	0
R7	300	150	633 832	362	48%	59 870	59 870	75 280	20%	1	0
R8	300	300	1 302 234	363	48%	122 505	122 505	126 015	3%	2	0
R9	300	450	1 790 174	350	47%	245 868	245 868	248 473	1%	1	0
R10	400	200	865 475	518	52%	84 224	84 224	99 484	15%	3	0
R11	400	400	1 715 442	445	45%	173 240	173 240	177 755	3%	3	0
R12	400	600	2 607 153	509	51%	284 969	284 969	288 406	1%	3	0
R13	500	250	1 051 104	605	50%	104 039	104 039	118 277	12%	6	0
R14	500	500	2 026 139	596	48%	178 562	178 562	186 443	4%	6	0
R15	500	750	3 001 778	599	47%	269 424	269 424	270 414	0%	5	0
R16	600	300	1 301 924	780	53%	111 304	111 304	128 960	14%	9	0
R17	600	600	2 505 231	745	51%	322 355	322 355	328 171	2%	7	0
R18	600	900	3 975 579	745	50%	555 574	555 574	558 786	1%	8	0
R19	700	350	1 594 247	956	53%	115 544	115 544	130 138	11%	11	0
R20	700	700	2 934 051	901	51%	358 421	358 421	366 774	2%	10	0
R21	700	1050	3 932 030	837	47%	413 551	413 551	417 155	1%	12	0
R22	800	400	1 755 904	987	49%	141 304	141 304	161 408	12%	14	0
R23	800	800	3 289 260	984	50%	314 218	314 218	323 928	3%	15	0
R24	800	1200	4 937 440	958	49%	588 442	588 442	592 415	1%	36	0
R25	900	450	1 799 520	1179	51%	161 878	161 846	182 088	11%	17	1
R26	900	900	3 668 963	1089	48%	414 915	414 915	424 240	2%	27	0
R27	900	1350	5 360 763	1105	49%	750 285	750 285	753 296	0%	49	0
R28	1000	500	2 236 205	1243	48%	167 809	167 809	186 625	10%	23	0
R29	1000	1000	4 010 665	1297	51%	378 959	378 959	384 965	2%	25	0
R30	1000	1500	6 010 931	1237	49%	728 177	728 177	732 494	1%	28	0
Mean					50%				6%	11	

The term 'D_ratio' represents the ratio between the number of drones and the number of lockers; 'F1' and 'F2' represent the objective values for model F1 and F2; 'Init_gap' denotes the initial gap between 'F1' and 'F2'; 'Opt*' is the optimal objective value.

The data for loaded and empty flight legs are also compared for SET 4, as shown in Table 4. The ultimate digraphs consist of multiple subgraphs, verifying that the underlying graph corresponding to a feasible solution is not necessarily connected. Empty flight legs must be added to ensure feasible for all the 30 instances. The ratio between numbers of empty and loaded flight legs varies from 54% to 77%, with an average of 69%. In comparison, the maximal distance ratio between empty and loaded flight legs is 16%, which is much lower than the former. This is because the objective is to minimize the distance rather than the number of empty flight legs.

Tests on R1–R30 demonstrate that the method can easily solve uniformly generated instances (including the largest instance containing 1000 sites, 1237 drones, and 1500 tasks) within a minute. Some 'Hard' instances (SET 5) are also generated to further verify the method's performance using expanded tests.

6.3. Tests on specially constructed 'Hard' instances

Instances are generated with specific numbers of subgraphs that are highly independent from each other. Each subgraph could be considered a "small island" (Applegate et al., 2011) with three sites, the locations of which are relatively fixed and formed an equilateral triangle. The size of each triangle is small such that the heights are all equal to 100. The underlying graph of each subgraph is connected and hence at least three flight legs are generated for each subgraph. All subgraphs are independent, meaning there are no loaded flight legs connecting any two subgraphs. The coordinate locations of all the subgraphs take values from the integer interval [0, 10 000]. Moreover, to make the instances challenging, whether a subgraph is active or not follows a uniform distribution. A total of 40 instances (R31–R70) are generated using this approach.

(1) General Results

Table 5 lists the statistical results. The number of active subgraphs is nearly equal to that of inactive subgraphs. The average numbers of active and inactive subgraphs across all 40 instances are 51 and 54, respectively, suggesting that the data are generated following a uniform distribution. It is evident that the number of subgraphs grows from 10 to 200 with increasing instance size. All the gaps are less than or equal to 1%, suggesting that a quasi optimal solution is found for each instance. The method can compute high-quality solutions for these challenging instances.

In the first stage, the number of nodes explored in this experiment is relatively small. In the computation of 13 out of 40 instances, only one node is explored. The largest number of explored nodes is 31 (R64). For comparison, the number of used cuts is much larger, with a maximum value of 3436 (R60) and a mean of 520. The reason why the B&C algorithm explores a small number of

Table 4

A comparison between the data of loaded and empty flight legs for SET 4.

ITC	Loaded flight leg		Empty flight leg		#Subgraph1	#Subgraph2	Ratio		
	#Flight Leg	Distance	#Flight Leg	Distance			Flight leg	Distance	Subgraph
R1	50	217 851	27	19 947	58	40	54%	9%	69%
R2	100	445 308	66	57 433	26	15	66%	13%	58%
R3	150	667 004	101	105 050	18	11	67%	16%	61%
R4	100	399 164	75	47 630	108	70	75%	12%	65%
R5	200	785 117	129	97 046	58	28	65%	12%	48%
R6	300	1 270 745	164	145 258	29	20	55%	11%	69%
R7	150	633 832	102	59 870	163	118	68%	9%	72%
R8	300	1 302 234	169	122 505	74	47	56%	9%	64%
R9	450	1 790 174	284	245 868	36	20	63%	14%	56%
R10	200	865 475	144	84 224	218	156	72%	10%	72%
R11	400	1 715 442	282	173 240	104	54	71%	10%	52%
R12	600	2 607 153	391	284 969	51	32	65%	11%	63%
R13	250	1 051 104	183	104 039	278	191	73%	10%	69%
R14	500	2 026 139	323	178 562	123	73	65%	9%	59%
R15	750	3 001 778	422	269 424	59	27	56%	9%	46%
R16	300	1 301 924	218	111 304	321	230	73%	9%	72%
R17	600	2 505 231	424	322 355	151	83	71%	13%	55%
R18	900	3 975 579	673	555 574	80	39	75%	14%	49%
R19	350	1 594 247	261	115 544	381	258	75%	7%	68%
R20	700	2 934 051	525	358 421	187	113	75%	12%	60%
R21	1050	3 932 030	675	413 551	92	56	64%	11%	61%
R22	400	1 755 904	309	141 304	438	297	77%	8%	68%
R23	800	3 289 260	565	314 218	211	129	71%	10%	61%
R24	1200	4 937 440	857	588 442	99	53	71%	12%	54%
R25	450	1 799 520	336	161 878	491	342	75%	9%	70%
R26	900	3 668 963	676	414 915	227	122	75%	11%	54%
R27	1350	5 360 763	1007	750 285	122	58	75%	14%	48%
R28	500	2 236 205	356	167 809	543	377	71%	8%	69%
R29	1000	4 010 665	742	378 959	259	150	74%	9%	58%
R30	1500	6 010 931	1083	728 177	119	59	72%	12%	50%
Mean	550	2 269 708	386	250 593	171	109	69%	11%	61%

Columns ‘#Subgraph1’ and ‘#Subgraph2’ represent the number of subgraphs in the initial and optimized digraphs, respectively.

nodes but employs a large number of cuts is likely due to the concise structure of the model. Since the primary coefficient matrix is totally unimodular, the linear relaxed solutions determined at each node tend to be integral. In addition, since the feasibility of a solution is guaranteed by dynamically added cut planes, rather than the primary coefficient matrix, infeasible solutions are more likely to be identified. In summary, solving a node tends to produce integral but infeasible solutions. The B&B framework is employed to tackle fractional solutions and the cuts are employed to tackle problem infeasible solutions. As a result, the algorithm explores only a small number of nodes but requires a large number of cuts.

The resulting CPU time is acceptable, with a maximum of 75 s. The times in Table 5 are slightly longer than those of Table 3, suggesting that instances in SET 5 are more challenging than those of SET 4. The proposed two-stage method successfully solves challenging instances with 600 sites, 319 drones, and 888 tasks.

(2) Sensitivity Analysis for Drone Percentage

The number of drones can be a key factor in determining the complexity of a problem, as demonstrated by two extreme cases. If there is only one drone in the delivery system, the LDDP becomes a variant of the TSP. In contrast, if each site is full of drones, strong “parking conflict” may also make the LDDP difficult to solve. It seems that both of these extreme cases are difficult to solve. Thus, a sensitivity analysis is conducted to determine the effects of drone quantities on solution efficiency. For clarity, ‘drone percent’ is defined as the ratio between the number of drones and the number of lockers. Different numbers of drones are used in the same instance to test solving efficiency. CPU time and the number of used cuts are recorded and are listed in Table 6 for ten selected instances. It is worth noting that nearly all computations result in near optimal solutions (Gap < 1%), with only four exceptions.

Both CPU time and the number of cuts change with the variation of the drone percent, which spans from 5% to 100% (Table 6). One of the most important findings in Table 6 is the significant trends in CPU time and the number of cuts, both of which show an overall decreasing trend with increasing drone percent. For most instances, solving instances with 5% to 10% drones requires significantly longer times. For example, solving instance R60 with 8% drones takes the longest calculation time (6419.8 s), roughly 1.78 h. In contrast, solving this instance with 20%–100% drones consume much shorter times of less than 10 s. Similar trends are observed for the number of cuts. For example, the algorithm solves all ten instances with 60%–100% drones to quasi optimality while rarely generating cuts. These observations suggest the algorithm struggled in instances with fewer drones, likely because these problems are more difficult to solve. A smaller number of drones will naturally result in greater routing flexibility (i.e., travel choices). In extreme cases, the LDDP becomes a variation of the TSP if only one drone is available for delivery. Increasing travel selection choices typically yields a massive discrete solution space. In the opposite extreme case, where all sites are full of drones,

Table 5
Statistical results for random instances in SET 5.

ITC	#Site	#Task	Dist	#Drone	DS_Ratio	#Subg	#ActSubg	#InaSubg	LB	UB	Gap	#Nodes	CPU(s)	#Cuts
R31	30	47	7512	28	93%	10	6	4	9740	9740	0%	1	0	5
R32	30	50	7994	18	60%	10	4	6	14 902	14 902	0%	1	0	7
R33	60	63	10 063	47	78%	20	12	8	5237	5237	0%	9	0	114
R34	60	91	14 483	44	73%	20	10	10	16 221	16 221	0%	1	0	11
R35	90	125	19 985	62	69%	30	15	15	24 046	24 046	0%	1	0	18
R36	90	134	21 608	37	41%	30	9	21	31 122	31 122	0%	1	0	22
R37	120	175	28 038	79	66%	40	17	23	31 321	31 321	0%	1	0	26
R38	120	182	29 202	75	63%	40	19	21	23 194	23 194	0%	1	0	22
R39	150	205	33 035	77	51%	50	23	27	28 587	28 587	0%	1	0	32
R40	150	208	33 340	107	71%	50	25	25	25 649	25 649	0%	1	0	28
R41	180	240	38 383	100	56%	60	23	37	36 050	36 050	0%	3	1	139
R42	180	241	38 878	99	55%	60	29	31	30 615	30 615	0%	1	1	35
R43	210	289	46 590	150	71%	70	35	35	33 596	33 596	0%	3	1	116
R44	210	302	48 600	130	62%	70	36	34	34 985	34 985	0%	1	1	35
R45	240	343	55 030	143	60%	80	36	44	39 957.5	39 966	0%	5	3	245
R46	240	350	56 843	131	55%	80	33	47	39 550	39 690	0%	5	4	297
R47	270	362	58 063	142	53%	90	41	49	42 493	42 601	0%	11	8	574
R48	270	375	60 073	216	80%	90	50	40	43 273	43 283	0%	3	3	123
R49	300	437	70 349	166	55%	100	42	58	47 503.5	47 504	0%	5	5	285
R50	300	440	71 008	170	57%	100	46	54	47 605	47 699	0%	15	15	882
R51	330	474	76 156	217	66%	110	58	52	43 383	43 412	0%	11	12	647
R52	330	478	76 602	191	58%	110	55	55	46 659.5	46 668	0%	3	5	194
R53	360	497	79 812	258	72%	120	67	53	43 427	43 427	0%	1	3	52
R54	360	541	86 901	199	55%	120	53	67	55 255	55 393	0%	9	13	641
R55	390	561	90 252	221	57%	130	60	70	54 563	54 563	0%	5	9	394
R56	390	574	91 790	286	73%	130	73	57	46 432.5	46 449	0%	3	7	155
R57	420	595	95 695	268	64%	140	68	72	54 288.5	54 478	0%	27	42	2104
R58	420	616	98 361	208	50%	140	54	86	58 869.5	59 264	1%	11	19	971
R59	450	615	98 574	315	70%	150	85	65	53 428	53 428	0%	1	4	66
R60	450	651	104 653	272	60%	150	72	78	56 211	56 753	1%	41	75	3436
R61	480	658	105 522	274	57%	160	75	85	60 983	61 319	1%	15	34	1262
R62	480	678	108 991	325	68%	160	79	81	57 130.5	57 163	0%	7	16	520
R63	510	730	116 618	309	61%	170	84	86	65 045.5	65 069	0%	5	16	440
R64	510	736	118 585	322	63%	170	85	85	62 918	63 271	1%	31	70	2581
R65	540	762	122 015	343	64%	180	81	99	68 521.5	68 551	0%	5	18	476
R66	540	775	124 556	384	71%	180	92	88	67 108.5	67 151	0%	3	12	249
R67	570	808	130 448	405	71%	190	101	89	63 235	63 235	0%	5	18	442
R68	570	822	131 478	384	67%	190	102	88	62 434	62 434	0%	5	24	467
R69	600	845	135 193	444	74%	200	106	94	67 520	67 520	0%	7	31	561
R70	600	888	142 495	319	53%	200	83	117	78 053.5	78 420	0%	19	69	2123
Mean					64%	51	54				0%	7.1	14	520

The term 'DS_Ratio' represents the ratio between numbers of drones and sites; 'Subg', 'ActSubg', and 'InaSubg' are abbreviations for 'Subgraph', 'Active Subgraph', and 'Inactive Subgraph', respectively.

there is less routing flexibility but more parking flexibility. The key point is that the difficulty caused by parking flexibility is less significant than the difficulty caused by routing flexibility. Hence, instances with fewer drones are likely more difficult to solve.

7. Conclusions

This article investigates the LDDP, which minimizes the total travel distance for drones engaged in parcel delivery among multiple lockers. The limited number of lockers becomes a shared resource for drones and produces a hybrid routing and parking problem. We find the sufficient and necessary conditions for feasible solutions. We develop a two-stage method for the problem. In the first stage, an efficient B&C algorithm is developed to solve a transformed problem. In the second stage, a heuristic is developed to construct practical schedules. The method is tested using both simulated and random instances. The simulated tests show that the required additional distance depends heavily on the type of delivery scenario. More lockers are suggested to be deployed at 'busy' sites to reduce the parking conflicts. The random tests show that the number of required cut planes is much greater than that of B&B nodes. This observation suggests that integral solutions are more likely to be found when solving a perfect formulation with cuts. The sensitivity analysis shows that the instances with fewer drones are more difficult to solve. This result indicates that the hardness caused by routing flexibility is stronger than that caused by parking flexibility. Overall, all the instances are solved to near optimal in acceptable CPU time, suggesting the method is promising for the problem.

The primary contribution of this study is the identification of sufficient and necessary conditions of feasible solutions. The condition is related to the b -transshipment theory in form but different from it in terms of purpose. We believe the conditions have the potential to be applied in some other areas, such as transportation management of trains. Moreover, an analysis of the proposed formulations demonstrates that row-doubled incidence matrices of a graph is also totally unimodular.

Table 6

Drone percent sensitivity analysis results for ten selected hard instances.

Drone	R32			R36			R40			R44			R48		
Percent	CPU(s)	Cuts	Gap	CPU(s)	Cuts	Gap	CPU(s)	Cuts	Gap	CPU(s)	Cuts	Gap	CPU(s)	Cuts	Gap
5%	0.1	7	0%	3.5	1634	1%	1.8	323	0%	15.0	2647	0%	9.2	1037	1%
6%	0.1	47	0%	0.3	25	0%	105.1	27 406	1%	1.0	58	0%	51.0	6525	0%
7%	0.1	27	0%	0.2	20	0%	1.1	87	0%	21.4	2848	0%	22.3	2331	0%
8%	0.1	9	0%	0.2	16	0%	4.6	738	0%	2.0	136	0%	10.4	808	0%
9%	0.1	4	0%	0.4	57	0%	1.8	206	0%	2.0	123	0%	4.3	241	1%
10%	0.1	4	0%	0.2	17	0%	0.5	29	0%	1.1	33	0%	1.5	53	0%
20%	0.1	0	0%	0.2	4	0%	0.5	7	0%	0.9	14	0%	1.5	19	0%
30%	0.1	0	0%	0.2	1	0%	0.5	3	0%	0.9	10	0%	1.4	9	0%
40%	0.1	0	0%	0.2	0	0%	0.5	2	0%	0.9	4	0%	2.0	2	0%
50%	0.1	0	0%	0.2	1	0%	0.5	0	0%	0.9	2	0%	1.4	2	0%
60%	0.1	0	0%	0.2	0	0%	0.4	0	0%	0.8	0	0%	1.7	0	0%
70%	0.1	0	0%	0.2	0	0%	0.5	0	0%	0.8	1	0%	1.4	0	0%
80%	0.1	0	0%	0.2	0	0%	0.5	0	0%	0.8	0	0%	1.3	0	0%
90%	0.1	0	0%	0.2	0	0%	0.6	0	0%	0.8	0	0%	1.4	0	0%
100%	0.1	0	0%	0.2	0	0%	0.5	0	0%	0.8	0	0%	1.8	0	0%

Drone	R52			R56			R60			R64			R68		
Percent	CPU(s)	Cuts	Gap	CPU(s)	Cuts	Gap	CPU(s)	Cuts	Gap	CPU(s)	Cuts	Gap	CPU(s)	Cuts	Gap
5%	5.2	288	0%	3889.3	14 187	1%	207.8	14 173	1%	N/A	N/A	N/A	N/A	N/A	N/A
6%	171.3	15 114	0%	65.7	4246	0%	24.8	1312	0%	321.6	13 008	100%	266.0	10 037	1%
7%	45.9	3900	1%	14.8	679	0%	76.6	4370	1%	202.7	9194	1%	N/A	N/A	N/A
8%	15.0	1031	1%	12.3	630	0%	6419.8	10 010	1%	3385.1	7266	0%	53.6	1907	0%
9%	7.4	322	0%	19.3	1052	0%	81.3	3911	1%	47.6	1517	0%	70.8	2359	0%
10%	2.8	63	0%	11.6	451	0%	14.2	448	0%	37.4	1087	1%	123.6	3550	0%
20%	2.5	29	0%	3.1	38	0%	4.8	31	0%	5.2	30	0%	7.5	41	0%
30%	2.3	13	0%	2.9	11	0%	4.5	8	0%	4.9	13	0%	7.3	11	0%
40%	2.1	7	0%	6.8	15	0%	3.9	4	0%	5.4	6	0%	6.9	6	0%
50%	2.1	2	0%	2.7	1	0%	4.0	3	0%	4.9	4	0%	7.0	0	0%
60%	2.3	0	0%	3.8	1	0%	3.7	1	0%	4.8	3	0%	7.2	3	0%
70%	2.0	0	0%	3.4	0	0%	4.5	1	0%	4.8	1	0%	6.8	0	0%
80%	2.2	0	0%	3.8	0	0%	3.7	0	0%	5.0	0	0%	7.5	0	0%
90%	2.1	0	0%	3.0	0	0%	4.4	0	0%	4.7	0	0%	7.0	1	0%
100%	1.9	0	0%	3.3	0	0%	3.5	0	0%	7.4	0	0%	5.9	0	0%

CRedit authorship contribution statement

Waiming Zhu: Conceptualization, Investigation, Methodology, Writing – original draft. **Xiaoxuan Hu:** Supervision, Project administration, Resources. **Jun Pei:** Writing – review & editing, Resources. **Panos M. Pardalos:** Supervision, Writing – review & editing.

Data availability

Data will be made available on request.

Acknowledgments

We thank the editors and two anonymous referees for their constructive suggestions which have enabled us to significantly improve the paper. We thank IBM ILOG CPLEX for providing free academic use. This study was funded by the National Natural Science Foundation of China under Grants 72001004, 72331004, 72071064, and 72188101.

Appendix A. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.trb.2024.102950>.

References

- Agatz, N., Bouman, P., Schmidt, M., 2018. Optimization approaches for the traveling salesman problem with drone. *Transp. Sci.* 52 (4), 965–981.
- Applegate, D.L., Bixby, R.E., Chvátal, V., Cook, W.J., 2011. The traveling salesman problem. In: *The Traveling Salesman Problem*. Princeton University Press.
- Avella, P., Boccia, M., Mannino, C., Vasilyev, I., 2017. Time-indexed formulations for the runway scheduling problem. *Transp. Sci.* 51 (4), 1196–1209.
- Barnhart, C., Boland, N.L., Clarke, L.W., Johnson, E.L., Nemhauser, G.L., Shenoi, R.G., 1998. Flight string models for aircraft fleet and routing. *Transp. Sci.* 32 (3), 208–220.
- Boland, N., Hewitt, M., Marshall, L., Savelsbergh, M., 2017. The continuous-time service network design problem. *Oper. Res.* 65 (5), 1303–1321.

- Carlsson, J.G., Song, S., 2018. Coordinated logistics with a truck and a drone. *Manage. Sci.* 64 (9), 4052–4069.
- Chen, X., Ulmer, M.W., Thomas, B.W., 2022. Deep Q-learning for same-day delivery with vehicles and drones. *European J. Oper. Res.* 298 (3), 939–952.
- Conforti, M., Cornuéjols, G., Zambelli, G., et al., 2014. *Integer Programming*, vol. 271, Springer.
- Cordeau, J.-F., Laporte, G., 2007. The dial-a-ride problem: Models and algorithms. *Ann. Oper. Res.* 153 (1), 29–46.
- Cordeau, J.-F., Stojković, G., Soumis, F., Desrosiers, J., 2001. Benders decomposition for simultaneous aircraft routing and crew scheduling. *Transp. Sci.* 35 (4), 375–388.
- Dantzig, G., Fulkerson, R., Johnson, S., 1954. Solution of a large-scale traveling-salesman problem. *J. Oper. Res. Soc. Am.* 2 (4), 393–410.
- Dayarian, I., Savelsbergh, M., Clarke, J.-P., 2020. Same-day delivery with drone resupply. *Transp. Sci.* 54 (1), 229–249.
- Dell'Amico, M., Montemanni, R., Novellani, S., 2020. Matheuristic algorithms for the parallel drone scheduling traveling salesman problem. *Ann. Oper. Res.* 289 (2), 211–226.
- Desaulniers, G., Desrosiers, J., Solomon, M.M., 2006. *Column Generation*, vol. 5, Springer Science & Business Media.
- Desaulniers, G., Errico, F., Irnich, S., Schneider, M., 2016. Exact algorithms for electric vehicle-routing problems with time windows. *Oper. Res.* 64 (6), 1388–1405.
- El-Adle, A., Ghoniem, A., Haouari, M., 2021. Parcel delivery by vehicle and drone. *J. Oper. Res. Soc.* 72 (2), 398–416.
- Faiz, T.I., Vogiatzis, C., Noor-E-Allah, M., 2024. Computational approaches for solving two-echelon vehicle and UAV routing problems for post-disaster humanitarian operations. *Expert Syst. Appl.* 237, 121473.
- Franco, M., 2016. DHL uses completely autonomous system to deliver consumer goods by drone. URL <https://newatlas.com/dhl-drone-delivery/43248/>.
- Gopalan, R., Talluri, K.T., 1998. The aircraft maintenance routing problem. *Oper. Res.* 46 (2), 260–271.
- Guo, L., Zheng, J., Liang, J., Wang, S., 2023. Column generation for the multi-port berth allocation problem with port cooperation stability. *Transp. Res. B* 171, 3–28.
- Ha, Q.M., Deville, Y., Pham, Q.D., Hà, M.H., 2018. On the min-cost traveling salesman problem with drone. *Transp. Res. C* 86, 597–621.
- Hall, N.G., Posner, M.E., 2001. Generating experimental data for computational testing with machine scheduling applications. *Oper. Res.* 49 (6), 854–865.
- Jeong, H.Y., Song, B.D., Lee, S., 2019. Truck-drone hybrid delivery routing: Payload-energy dependency and no-fly zones. *Int. J. Prod. Econ.* 214, 220–233.
- Jia, S., Li, C.-L., Xu, Z., 2020a. A simulation optimization method for deep-sea vessel berth planning and feeder arrival scheduling at a container port. *Transp. Res. B* 142, 174–196.
- Jia, S., Meng, Q., Kuang, H., 2022. Equitable vessel traffic scheduling in a seaport. *Transp. Sci.* 56 (1), 162–181.
- Jia, S., Wu, L., Meng, Q., 2020b. Joint scheduling of vessel traffic and pilots in seaport waters. *Transp. Sci.* 54 (6), 1495–1515.
- Jian, S., Liu, W., Wang, X., Yang, H., Waller, S.T., 2020. On integrating carsharing and parking sharing services. *Transp. Res. B* 142, 19–44.
- Laporte, G., 2009. Fifty years of vehicle routing. *Transp. Sci.* 43 (4), 408–416.
- Ledvina, K., Qin, H., Simchi-Levi, D., Wei, Y., 2022. A new approach for vehicle routing with stochastic demand: Combining route assignment with process flexibility. *Oper. Res.*
- Lei, D., Chen, X., 2022. An improved variable neighborhood search for parallel drone scheduling traveling salesman problem. *Appl. Soft Comput.* 127, 109416.
- Li, H., Chen, J., Wang, F., Bai, M., 2021. Ground-vehicle and unmanned-aerial-vehicle routing problems from two-echelon scheme perspective: A review. *European J. Oper. Res.* 294 (3), 1078–1095.
- Li, C.-L., Pang, K.-W., 2011. An integrated model for ship routing and berth allocation. *Int. J. Ship. Transp. Logist.* 3 (3), 245–260.
- Li, Q., Xia, J., Ge, F., Lu, Q., Zhang, M., 2022. Blood delivery by drone: A faltering step in a promising direction. *Lancet Global Health* 10 (8), e1098.
- Li, F., Yang, D., Wang, S., Weng, J., 2019. Ship routing and scheduling problem for steel plants cluster alongside the Yangtze River. *Transp. Res. E* 122, 198–210.
- Liang, J., 2019. First license for drone delivery test-running granted in East China. *en.people.cn*, URL <http://en.people.cn/n3/2019/1017/c90000-9623668.html>.
- Liang, Z., Chaovalitwongse, W.A., 2013. A network-based model for the integrated weekly aircraft maintenance routing and fleet assignment problem. *Transp. Sci.* 47 (4), 493–507.
- Liang, Z., Feng, Y., Zhang, X., Wu, T., Chaovalitwongse, W.A., 2015. Robust weekly aircraft maintenance routing problem and the extension to the tail assignment problem. *Transp. Res. B* 78, 238–259.
- Liu, W., Sun, X., 2022. Energy-aware and delay-sensitive management of a drone delivery system. *Manuf. Serv. Oper. Manag.* 24 (3), 1294–1310.
- Liu, W., Yang, H., Yin, Y., 2014. Expirable parking reservations for managing morning commute with parking space constraints. *Transp. Res. C* 44, 185–201.
- Lübbecke, M.E., Desrosiers, J., 2005. Selected topics in column generation. *Oper. Res.* 53 (6), 1007–1023.
- Meng, Q., Wang, S., Lee, C.-Y., 2015. A tailored branch-and-price approach for a joint tramp ship routing and bunkering problem. *Transp. Res. B* 72, 1–19.
- Murray, C.C., Chu, A.G., 2015. The flying sidekick traveling salesman problem: Optimization of drone-assisted parcel delivery. *Transp. Res. C* 54, 86–109.
- Nguyen, M.A., Dang, G.T.-H., Hà, M.H., Pham, M.-T., 2022. The min-cost parallel drone scheduling vehicle routing problem. *European J. Oper. Res.* 299 (3), 910–930.
- Nisingizwe, M.P., Ndishimye, P., Swaibu, K., Nshimiyimana, L., Karame, P., Dushimiyimana, V., Musabyimana, J.P., Musanabaganwa, C., Nsanzimana, S., Law, M.R., 2022. Effect of unmanned aerial vehicle (drone) delivery on blood product delivery time and wastage in Rwanda: A retrospective, cross-sectional study and time series analysis. *Lancet Global Health* 10 (4), e564–e569.
- Otto, A., Agatz, N., Campbell, J., Golden, B., Pesch, E., 2018. Optimization approaches for civil applications of Unmanned Aerial Vehicles (UAVs) or aerial drones: A survey. *Networks* 72 (4), 411–458.
- Pang, K.-W., Liu, J., 2014. An integrated model for ship routing with transshipment and berth allocation. *IIE Trans.* 46 (12), 1357–1370.
- Paradiso, R., Roberti, R., Laganá, D., Dullaert, W., 2020. An exact solution framework for multitrip vehicle-routing problems with time windows. *Oper. Res.* 68 (1), 180–198.
- Pessoa, A.A., Poss, M., Sadykov, R., Vanderbeck, F., 2021. Branch-cut-and-price for the robust capacitated vehicle routing problem with knapsack uncertainty. *Oper. Res.* 69 (3), 739–754.
- Poikonen, S., Golden, B., 2020. The mothership and drone routing problem. *INFORMS J. Comput.* 32 (2), 249–262.
- Poikonen, S., Golden, B., Wasi, E.A., 2019. A branch-and-bound approach to the traveling salesman problem with a drone. *INFORMS J. Comput.* 31 (2), 335–346.
- Raj, R., Lee, D., Lee, S., Walteros, J., Murray, C., 2021. A branch-and-price approach for the parallel drone scheduling vehicle routing problem. Available at SSRN 3879710.
- Roberti, R., Ruthmair, M., 2021. Exact methods for the traveling salesman problem with drone. *Transp. Sci.* 55 (2), 315–335.
- Ruther, S., Boland, N., Engineer, F.G., Evans, I., 2017. Integrated aircraft routing, crew pairing, and tail assignment: Branch-and-price with many pricing problems. *Transp. Sci.* 51 (1), 177–195.
- Sah, B., Gupta, R., Bani-Hani, D., 2021. Analysis of barriers to implement drone logistics. *Int. J. Logist. Res. Appl.* 24 (6), 531–550.
- Saleu, R.G.M., Deroussi, L., Feillet, D., Grangeon, N., Quilliot, A., 2022. The parallel drone scheduling problem with multiple drones and vehicles. *European J. Oper. Res.* 300 (2), 571–589.
- Sandhu, R., Klabjan, D., 2007. Integrated airline fleet and crew-pairing decisions. *Oper. Res.* 55 (3), 439–456.
- Schrijver, A., et al., 2003. *Combinatorial Optimization: Polyhedra and Efficiency*, vol. 24, Springer.
- Tamke, F., Buscher, U., 2021. A branch-and-cut algorithm for the vehicle routing problem with drones. *Transp. Res. B* 144, 174–203.
- Tiniç, G.O., Karasan, O.E., Kara, B.Y., Campbell, J.F., Ozel, A., 2023. Exact solution approaches for the minimum total cost traveling salesman problem with multiple drones. *Transp. Res. B* 168, 81–123.
- Van den Akker, J., Hurkens, C.A., Savelsbergh, M.W., 2000. Time-indexed formulations for machine scheduling problems: Column generation. *INFORMS J. Comput.* 12 (2), 111–124.

- Vogiatzis, C., Camur, M.C., 2019. Identification of essential proteins using induced stars in protein–protein interaction networks. *INFORMS J. Comput.* 31 (4), 703–718.
- Vogiatzis, C., Walteros, J.L., 2018. Integer programming models for detecting graph bipartitions with structural requirements. *Networks* 71 (4), 432–450.
- Wang, Y., Buchanan, A., Butenko, S., 2017. On imposing connectivity constraints in integer programs. *Math. Program.* 166 (1), 241–271.
- Wang, S., Meng, Q., Sun, Z., 2013. Container routing in liner shipping. *Transp. Res. E* 49 (1), 1–7.
- Wang, Z., Sheu, J.-B., 2019. Vehicle routing problem with drones. *Transp. Res. B* 122, 350–364.
- Wang, T., Wang, X., Meng, Q., 2018. Joint berth allocation and quay crane assignment under different carbon taxation policies. *Transp. Res. B* 117, 18–36.
- West, D.B., et al., 2001. *Introduction to Graph Theory*, vol. 2, Prentice hall Upper Saddle River.
- Wu, L., Adulyasak, Y., Cordeau, J.-F., Wang, S., 2022. Vessel service planning in seaports. *Oper. Res.* 70 (4), 2032–2053.
- Wu, L., Wang, S., Laporte, G., 2021. The robust bulk ship routing problem with batched cargo selection. *Transp. Res. B* 143, 124–159.
- Yang, H., Liu, W., Wang, X., Zhang, X., 2013. On the morning commute problem with bottleneck congestion and parking space constraints. *Transp. Res. B* 58, 106–118.
- Yang, Y., Yan, C., Cao, Y., Roberti, R., 2023. Planning robust drone-truck delivery routes under road traffic uncertainty. *European J. Oper. Res.* 309 (3), 1145–1160.
- Zhen, L., Gao, J., Tan, Z., Wang, S., Baldacci, R., 2022a. Branch-price-and-cut for trucks and drones cooperative delivery. *IIE Trans.* 1–17.
- Zhen, L., Zhuge, D., Wang, S., Wang, K., 2022b. Integrated berth and yard space allocation under uncertainty. *Transp. Res. B* 162, 1–27.
- Zhou, H., Qin, H., Cheng, C., Rousseau, L.-M., 2023. An exact algorithm for the two-echelon vehicle routing problem with drones. *Transp. Res. B* 168, 124–150.